Relatório da atividade individual de Pinia

Projeto: CTI Insights
Aluno: _____
RA: _____
Turma: _____
GitHub: https://github.com/recheraphael-Re/CTI_SENAI

O projeto utiliza Vue 3 e Pinia para receber uma planilha, armazenar os registros temporariamente, validar os dados e apresentar um relatório. O fluxo implementado é **planilha → tela de upload → store do Pinia → tela de relatório**. Este documento descreve os cinco pontos solicitados na atividade individual.

Estrutura do projeto e inicialização

Os arquivos têm responsabilidades diferentes. A tela recebe a interação do usuário, a store coordena a operação e compartilha os resultados, e os utilitários executam as regras de leitura e validação.

```
CTI_SENAI/
├── Data/                          Bases utilizadas na demonstração
└── FrontEnd/
    ├── src/
    │   ├── main.js                Inicialização do Vue e do Pinia
    │   ├── router/index.js        Rotas de upload e relatório
    │   ├── stores/uploadStore.js  Estado, getters e actions
    │   ├── utils/planilha.js      Leitura, tratamento e validação
    │   ├── utils/formato.js       Conversão e formatação de valores
    │   └── views/
    │       ├── UploadView.vue     Seleção e processamento
    │       └── RelatoriosView.vue Exibição dos resultados
    └── tests/                    Verificação das regras e da store
```

Trecho de `FrontEnd/src/main.js`:

```
const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

`createApp` cria a aplicação Vue. `app.use(createPinia())` registra o gerenciador de estado antes de montar a interface. O router define qual tela será exibida. `mount` conecta a aplicação ao elemento HTML com identificador `app`.

Os exemplos a seguir são trechos do projeto. Quando um trecho contém apenas parte de uma função, ele é identificado como recorte e não deve ser interpretado como uma implementação completa.

1 O que foi armazenado no state

A store está em `FrontEnd/src/stores/uploadStore.js`, definida por `defineStore('upload', ...)`. Foi utilizado o formato Setup Store: `ref` e `shallowRef` representam o state, `computed` representa os getters e as funções representam as actions.

No state foram armazenados o arquivo selecionado (`arquivo`), os dados originais da planilha (`dadosOriginais`), os registros tratados (`clientes`), as ocorrências de erro (`erros`) e o resumo da validação (`resumo`). Cada cliente mantém sua linha na planilha, os campos tratados, a lista de erros e a indicação de registro válido ou inválido.

Também são armazenados o nome do arquivo (`nomeArquivo`), a data do processamento (`dataUpload`), a quantidade de colunas (`totalColunas`), o período do faturamento (`periodoFaturamento`), o estado de carregamento (`carregando`) e a mensagem de falha da operação (`erro`). `erros` reúne problemas dos registros; `erro` informa falhas que impedem a leitura.

Os resultados ficam em memória e são gravados em `sessionStorage` para recuperação após atualizar a página na mesma sessão. O objeto File não é restaurado. A planilha não é enviada a um backend.

1.1 Declaração e recuperação do estado

Recorte de `FrontEnd/src/stores/uploadStore.js`, dentro de `defineStore`:

```
export const useUploadStore = defineStore('upload', () => {
  const sessao = lerDaSessao()
  // File permanece em memória: não pode ser restaurado pelo JSON da sessão.
  const arquivo = shallowRef(null)
  const dadosOriginais = ref(sessao?.dadosOriginais ?? [])
  const clientes = ref(sessao?.clientes ?? [])
  const erros = ref(sessao?.erros ?? [])
  const resumo = ref(sessao?.resumo ?? { ...RESUMO_VAZIO })
```

`upload` é o identificador da store. `lerDaSessao()` tenta recuperar a importação anterior. O operador `?.` permite consultar uma sessão que pode não existir; `?? []` fornece uma lista vazia quando o valor recuperado é nulo ou indefinido. `ref` torna esses dados reativos, de modo que as telas acompanhem suas alterações. `shallowRef` mantém a referência do File sem converter profundamente suas propriedades.

O estado e as funções públicas são incluídos no `return` ao final da definição da store. Dentro dela, os refs são acessados por `.value`, como em `clientes.value`. Nos consumidores, o Pinia permite acessar o valor diretamente por `store.clientes`.

1.2 Dados originais e resultado tratado

Em `utils/planilha.js`, os dados originais são preservados antes da validação. Depois, cada registro recebe o resultado:

```
registro.erros = validar(registro, bruto, listas, vistos, mapa.email !== undefined)
registro.valida = registro.erros.length === 0
registro.ativo = registro.status === 'ATIVO'

for (const erro of registro.erros) erros.push({ linha: registro.linha, ...erro })
clientes.push(registro)
```

`registro.erros` contém os problemas de um único cliente. `valida` só é verdadeiro se essa lista estiver vazia. A lista geral `erros` recebe cada ocorrência com a linha de origem, permitindo agrupar erros da planilha inteira. O operador `...erro` copia tipo, campo e descrição para esse objeto. Mesmo os clientes inválidos são mantidos para que possam ser consultados no relatório.

2 Quais getters foram criados e o que calculam

- `totalLinhas`, `totalValidas` e `totalComErro`: total de registros importados, quantidade válida e quantidade com erro.
- `linhasValidas` e `linhasComErro`: listas de registros separados pelo resultado da validação.
- `totalErros` e `errosPorTipo`: total de ocorrências e contagem agrupada por tipo de erro.
- `dadosTratados`, `totalClientes` e `clientesNivelA`: acesso à base tratada, quantidade de clientes válidos e quantidade válida de nível A.
- `temDados`, `usandoExemplo` e `clientesExibidos`: indicação de base importada ou demonstrativa e seleção da lista exibida nos indicadores.
- `clientesValidos` e `faturamentoTotal`: lista válida usada nos indicadores e soma de seu faturamento.

- `indicadores` e `faturamentoPorSegmento`: clientes válidos, faturamento médio, registros ativos e soma por segmento dos dez maiores totais.
- `rotuloFaturamento`: texto que identifica o período mensal, anual ou não informado.

Uma linha pode apresentar vários erros; por isso, registros com erro e ocorrências podem ter quantidades diferentes. O relatório utiliza apenas registros importados. Os indicadores podem mostrar uma amostra enquanto nenhuma planilha foi carregada.

2.1 Filtragem e contagem reativas

Trecho de `FrontEnd/src/stores/uploadStore.js`:

```
const totalLinhas = computed(() => clientes.value.length)
const linhasValidas = computed(() => clientes.value.filter(c => c.valida))
const linhasComErro = computed(() => clientes.value.filter(c => !c.valida))
const totalValidas = computed(() => linhasValidas.value.length)
const totalComErro = computed(() => linhasComErro.value.length)
```

`filter` cria uma lista com os registros que atendem à condição. `c.valida` seleciona registros válidos; `!c.valida` seleciona os inválidos. `length` fornece a quantidade. Como as expressões estão em `computed`, seus resultados acompanham a base atualizada sem exigir que a tela refaça a contagem manualmente.

Exemplo: se há três registros e apenas um deles tem `valida: false`, os getters retornam `totalLinhas = 3`, `totalValidas = 2` e `totalComErro = 1`.

2.2 Contagem por tipo de erro

Trecho da mesma store:

```
const errosPorTipo = computed(() => {
  const contagem = new Map()
  for (const e of erros.value) contagem.set(e.tipo, (contagem.get(e.tipo) ?? 0) + 1)
  return [...contagem.entries()]
    .sort((a, b) => b[1] - a[1])
    .map(([tipo, quantidade]) => ({ tipo, quantidade }))
})
```

O `Map` associa cada tipo à quantidade encontrada. Para cada ocorrência, a contagem anterior é incrementada, começando em zero. `sort` coloca os tipos mais frequentes primeiro e `map` devolve objetos com os campos que a interface utiliza. Dois erros de e-mail e um campo vazio produzem duas entradas: `EMAIL_INVALIDO` com quantidade 2 e `CAMPO_OBRIGATORIO` com quantidade 1.

3 Quais actions foram implementadas

- `selecionarArquivo(novo)`: verifica a extensão XLSX, XLS ou CSV e guarda o arquivo. Bloqueia uma nova seleção durante o processamento.
- `validarArquivo()`: confere se existe um arquivo selecionado com extensão aceita.
- `processarPlanilha()`: inicia a importação do arquivo selecionado.
- `importarPlanilha(arquivoSelecionado)`: controla o carregamento, executa a leitura e a validação, atualiza o state, salva os resultados na sessão e informa falhas. Retorna `true` em caso de sucesso e `false` em caso de falha ou operação descartada.
- `limpar()`: remove dados e erros, apaga a persistência da sessão e invalida resultados de operações anteriores ainda pendentes.

As regras foram organizadas em `FrontEnd/src/utils/planilha.js`. A action de importação executa esse módulo e armazena os resultados na store. As telas consomem estado e getters com `storeToRefs`, preservando a reatividade.

3.1 Como a tela inicia o processamento

Trecho de `FrontEnd/src/views/UploadView.vue`:

```
function receberArquivo(novo) {
  store.selecionarArquivo(novo)
}

async function processar() {
  await store.processarPlanilha()
}
```

A tela encaminha o arquivo à store em vez de criar outra lista de clientes. A função `processar` aguarda a action assíncrona. Na store, a ligação com a importação é:

```
async function processarPlanilha() {
  return importarPlanilha(arquivo.value)
}
```

3.2 Leitura e atualização do state

Recorte do bloco `try` de `importarPlanilha`, na store:

```
const lido = await lerPlanilha(arquivoSelecionado)
if (atual !== operacao) return false
dataUpload.value = new Date().toISOString()
periodoFaturamento.value = lido.periodoFaturamento
totalColunas.value = lido.totalColunas
clientes.value = lido.clientes
dadosOriginais.value = lido.dadosOriginais
erros.value = lido.erros
resumo.value = lido.resumo
nomeArquivo.value = arquivoSelecionado.name
```

`await` espera a leitura terminar antes de atualizar o estado. `lido` reúne os resultados devolvidos pelo utilitário. As atribuições atualizam a base compartilhada, os getters acompanham as mudanças e o relatório pode consultar os mesmos resultados. A comparação com `operacao` impede que uma leitura antiga repovoe a store depois de a base ter sido limpa.

Recorte do tratamento de falhas da mesma action:

```
} catch (e) {
  if (atual !== operacao) return false
  erro.value = e.message ?? 'Não foi possível ler a planilha.'
  return false
} finally {
  if (atual === operacao) carregando.value = false
}
```

`catch` transforma a falha em uma mensagem para a interface. `finally` encerra o carregamento da operação vigente tanto no sucesso quanto na falha. Isso evita deixar o botão preso no estado de processamento.

4 Quais validações foram realizadas

- **Campos obrigatórios:** registra `CAMPO_OBRIGATORIO` para código, cliente, consultor, serviço, segmento, nível e faturamento vazios.
- **Duplicidade:** registra `DUPLICIDADE` a partir da segunda ocorrência do mesmo código de cliente.
- **E-mail:** quando a coluna `email` existe, registra `EMAIL_INVALIDO` para formato inadequado e `CAMPO_OBRIGATORIO` para e-mail vazio. A verificação é de formato, sem comprovar a existência da caixa postal.

- **Dados fora do padrão:** registra `DOMINIO_INVALIDO` para nível ou status preenchido fora das listas de referência. A conferência usa o texto original, mesmo que a apresentação seja padronizada.
- **Números:** registra `NUMERO_INVALIDO` para conteúdo não numérico preenchido e `VALOR_NEGATIVO` para faturamento ou valor de serviço negativo.
- **Datas:** registra `DATA_INVALIDA` para data preenchida que não pode ser interpretada, incluindo datas inexistentes como 31/02/2026.
- **Espaços e textos:** remove espaços nas extremidades, reduz espaços repetidos em cliente e consultor e padroniza grafias de segmento, serviço, nível e status. Por exemplo, `IND.` é apresentado como `Indústria`. Os valores originais são preservados.
- **Estrutura do arquivo:** interrompe a leitura se faltarem colunas obrigatórias, houver cabeçalhos ambíguos ou não forem encontrados registros.

A coluna de e-mail é opcional porque a base oficial não a contém. Quando presente, todos os e-mails da coluna são verificados. O arquivo `Data/Exemplo_Validacao_Email.csv` permite demonstrar esse caso. Espaços e grafias tratáveis são corrigidos para apresentação e não geram, por si só, uma ocorrência de erro. Linhas com erros ficam fora dos indicadores.

4.1 Registro de um erro e verificação de duplicidade

Recorte da função `validar`, em `FrontEnd/src/utils/planilha.js`:

```
const erros = []
const acusar = (tipo, campo, descricao) => erros.push({ tipo, campo, descricao })
```

`acusar` padroniza o formato de todas as ocorrências. A mesma função registra campos vazios, duplicidade, formato de e-mail e outros problemas. Assim, a interface consegue exibir qualquer regra usando os mesmos campos.

Trecho da regra de duplicidade:

```
if (registro.id) {
  if (vistos.has(registro.id)) {
    acusar('DUPLICIDADE', 'codigo_cliente', `Código duplicado: ${registro.id}.`)
  } else {
    vistos.set(registro.id, registro.linha)
  }
}
```

`vistos` é um `Map` compartilhado durante a leitura das linhas do arquivo. A primeira ocorrência guarda o código; a próxima encontra esse código com `has` e registra o erro. Por isso, a segunda linha repetida é marcada, enquanto a primeira permanece como referência. A comparação utiliza o código após remover espaços das extremidades.

4.2 Validação de e-mail

Trecho da função `validar`:

```
if (temEmail) {
  if (!registro.email) acusar('CAMPO_OBRIGATORIO', 'email', 'E-mail vazio.')
  else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(registro.email)) {
    acusar('EMAIL_INVALIDO', 'email', `E-mail em formato inválido: ${registro.email}.`)
  }
}
```

`temEmail` informa se a coluna foi encontrada. A expressão regular exige partes antes e depois do `@`, um ponto na parte do domínio e ausência de espaços. `test` retorna verdadeiro ou falso. `contato@empresa.com` passa; `sem-arroba.com` falha; um valor vazio recebe uma ocorrência de campo obrigatório. Essa regra verifica um formato básico, sem consultar servidores de e-mail.

4.3 Tratamento de espaços

Trecho do utilitário:

```
function limparTexto(valor) {
  return String(valor ?? '').trim().replace(/\s+/g, ' ')
}
```

`String` converte o valor em texto, `trim` remove espaços no início e no fim, e `replace` reduz sequências de espaços em branco a um único espaço. Por exemplo, `"  Empresa  Um  "` torna-se `"Empresa Um"`. Os dados originais permanecem em `dadosOriginais`; o texto tratado é utilizado no registro exibido. Esse tratamento não é registrado como erro.

5 Como os resultados são apresentados no sistema

Na tela `/app/upload`, o usuário seleciona ou arrasta a planilha e clica em **Enviar e processar**. O sistema mostra o nome do arquivo, a aba importada, as contagens da validação, as ocorrências por tipo e uma prévia dos dez primeiros registros tratados. Falhas de leitura aparecem na própria tela.

Na tela `/app/relatorios`, são apresentados o nome do arquivo, a data do processamento, o total de registros, a quantidade válida, a quantidade com erro e a contagem de cada tipo de erro. A tabela informa a linha de origem e, para cada problema, o campo afetado, o tipo e a descrição. Registros inválidos recebem destaque visual; os demais mostram **Sem erros**.

O relatório permite buscar clientes e filtrar somente linhas com erro. Sem importação, orienta o usuário a enviar uma planilha. O botão **Descartar planilha**, na tela de upload, limpa os resultados.

5.1 Acesso à mesma store no relatório

Trecho de `FrontEnd/src/views/RelatoriosView.vue`:

```
const store = useUploadStore()
const { clientes: clientesExibidos, usandoExemplo, resumo } = storeToRefs(store)
```

`useUploadStore()` obtém a store já utilizada pelo upload. `storeToRefs` permite desestruturar estado e getters mantendo a reatividade. `clientes: clientesExibidos` apenas dá outro nome à referência local: não cria uma cópia da base e não acessa o getter demonstrativo `clientesExibidos`. O relatório continua trabalhando com `clientes`, os registros importados.

5.2 Exibição do campo e da descrição

Recorte do template de `RelatoriosView.vue`, dentro da célula de problemas:

```
<ul v-if="cliente.erros.length"
    class="list-disc space-y-1 pl-4 text-rose-600 dark:text-rose-400">
  <li v-for="(erro, indice) in cliente.erros" :key="indice">
    <strong>{{ erro.campo }}</strong> ({{ erro.tipo }}): {{ erro.descricao }}
  </li>
</ul>
<span v-else class="text-emerald-600">Sem erros</span>
```

`v-if` escolhe entre apresentar os problemas e a mensagem de registro válido. `v-for` cria um item para cada ocorrência. As interpolações `{{ ... }}` exibem o campo, o tipo e a descrição recebidos da validação. O resultado de um e-mail incorreto aparece como: **email (EMAIL_INVALIDO): E-mail em formato inválido: sem-arroba.com.** A coluna Linha identifica sua posição no arquivo.

5.3 Evidência em um teste do projeto

Trecho de `FrontEnd/tests/dados.test.js`, do teste da base oficial:

```
assert.deepEqual(resultado.resumo, {
  aba: 'dados_importacao', linhas: 300,
  validas: 265, comErro: 35, ocorrencias: 37
})
```

O exemplo acima apenas distribui em mais linhas a mesma asserção do arquivo. `assert.deepEqual` compara o objeto produzido pela leitura com o resultado esperado. O teste também compara linha, tipo e campo das ocorrências com o gabarito da planilha; portanto, verifica tanto as quantidades quanto a identificação dos problemas. Se uma regra deixar de detectar uma ocorrência esperada, o teste falha.

5.4 Verificação da entrega

A base oficial `Data/CTI_Insights_Base_Validacao_300_Registros.xlsx` produz **300 registros, 265 válidos, 35 com erro e 37 ocorrências**, conferidos pelo teste que compara o resultado com o gabarito da planilha. E-mail e limpeza de espaços possuem um teste específico. Nesta revisão, os oito testes passaram e a compilação foi concluída.

Comandos na pasta `FrontEnd`:

```
npm.cmd install
npm.cmd run dev
node --test --test-isolation=none tests/*.test.js
npm.cmd run build
```

Importando `Data/Exemplo_Validacao_Email.csv`, o resultado esperado é **4 registros, 1 válido e 3 com erro**, sendo duas ocorrências de `EMAIL_INVALIDO` e uma de `CAMPO_OBRIGATORIO`. No registro válido, os espaços excedentes de cliente e consultor são removidos.

Entrega: preencher a identificação deste documento, entregar o Word e informar o link do GitHub acima.